

Hermes Agent Setup Guide

How we set up Hermes Agent for Suki Pulse: the CLI, the Desktop app, a Telegram bot, local or VPS hosting, and the MCP servers we connected. Written for a first-time setup.

Suki Pulse team · October 2026 · Hermes Agent v0.21. Commands change between releases, so check the official docs when something does not match: hermes-agent.nousresearch.com/docs

Contents

1. Install the CLI
2. Windows: run it in WSL
3. The Desktop app
4. A Telegram bot
5. Local or VPS hosting
6. Local hosting that stays up
7. Choosing a VPS
8. Setting up the VPS
9. How MCP servers work
10. GitHub
11. Vercel, Neon and Supabase
12. Google Workspace
13. Final check

Order matters. Do section 1 first. The Desktop app and the Telegram gateway both run on the same backend and the same `~/hermes` folder, so everything you set up in the CLI carries over.

1. Install the CLI

- 1. **Run the installer** (macOS, Linux or WSL). It brings its own Python environment through `uv`, so there is nothing else to install first.

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash
```
- 2. **Reload the shell**: open a new terminal, or `source ~/.bashrc` (`~/.zshrc` on macOS).
- 3. **Check it is on your PATH**: `hermes --version`. "command not found" means the shell has not reloaded yet.
- 4. **Run the setup wizard**: `hermes setup`. It asks about the model, terminal permissions and the gateway. You can run it again later to change any answer.
- 5. **Pick a model**: `hermes model`. If you already pay for a chat subscription (Claude, ChatGPT), choose its sign-in option instead of pasting an API key.
- 6. **Health check**: `hermes doctor`, and `hermes doctor --fix` for the common problems.
- 7. **First chat**: `hermes` for an interactive session, or `hermes chat -q "hello"` for one question.

Path	What it holds
<code>~/.hermes/config.yaml</code>	Settings only. No secrets.
<code>~/.hermes/.env</code>	API keys and tokens only.
<code>~/.hermes/skills/</code>	Installed skills.
<code>~/.hermes/logs/</code>	Gateway and error logs.
<code>~/.hermes/state.db</code>	Session history.

Change settings with `hermes config set KEY VALUE` rather than editing `config.yaml` by hand. One wrong indent there can break the whole agent without a clear error.

2. Windows: run it in WSL

We ran Hermes inside Ubuntu 24.04 on WSL2, so every Linux command in this guide works unchanged and the laptop matches a Linux VPS. Three problems came up on a fresh Ubuntu, each with a one-line fix:

Symptom	Cause	Fix
Installer stops at <code>npm install</code> failed	Ubuntu is missing <code>libatomic1</code> , so the bundled Node cannot start	<code>sudo apt install -y libatomic1</code> , then rerun the installer
<code>hermes desktop</code> first build stops at <code>gyp ERR! not ok</code>	No C compiler	<code>sudo apt install -y build-essential</code>
Background jobs get killed	WSL ran out of memory with many browser tabs and agent sessions open	Close what you do not need before the first desktop build

Install WSL with `wsl --install` from an admin terminal. The Desktop app then opens as a Linux window on Windows.

3. The Desktop app

1. Finish section 1. The app is a window onto the same backend and needs no separate setup.
2. Start it with `hermes desktop` (or `hermes gui`). The first start takes a few seconds while the backend boots.
3. Your model from `hermes setup` is already selected. If none is set, the app asks you to pick one.
4. **Ctrl+K** (Cmd+K on macOS) opens the command palette: switch sessions, open settings, run actions.
5. Optional: change the theme with `hermes config set display.skin`. Every open window repaints within a second.
6. Optional: `hermes profile create` gives a fully separate config, skill set and memory, for example to keep work and personal setups apart.

4. A Telegram bot

The gateway runs the same agent inside Telegram with its full toolset: it can run commands and edit files from your phone. That makes step 4 below the important one.

1. In Telegram, message `@BotFather` and send `/newbot`. Pick a name and a username ending in `bot`. Save the token it sends back.
2. Run `hermes gateway setup`, choose Telegram and paste the token. Or add it to `~/.hermes/.env` yourself:

```
TELEGRAM_BOT_TOKEN=<your bot token>
```
3. Message `@userinfobot` to get your numeric Telegram user ID (a number like `123456789`).
4. **Lock the bot to yourself.** Enter that ID as the allowed user during gateway setup. An open bot lets anyone who finds it run commands on your machine.
5. Start it with `hermes gateway start`, or `hermes gateway run` to watch the logs while testing.
6. Check it: `hermes gateway status`, or `/platform list` inside a chat.
7. Send your bot any message. It should reply.

Command	Use
<code>hermes gateway restart</code>	After a config change
<code>hermes gateway stop</code>	Stop the bot
<code>hermes gateway status</code>	Connection health
<code>~/.hermes/logs/gateway.log</code>	First place to look when it misbehaves

5. Local or VPS hosting

This decides whether the bot answers only while your computer is on, or around the clock.

	Your own computer	A VPS
Cost	Free	A few dollars a month, or a free tier
Setup	Nothing extra: no server, no SSH, no firewall	SSH, a Linux user, a firewall
Uptime	Down when the machine sleeps, shuts down or loses internet	Always on, independent of your laptop
Inbound webhooks	Need a tunnel or port forwarding	Static public IP
Access	Your local files, apps and dev setup	Only what you install there
Upkeep	None beyond your own machine	Security updates, backups, monitoring

Start local. Move to a VPS when you need a static IP for webhooks, or when a team depends on the bot.

6. Local hosting that stays up

1. Finish sections 1 and 4 and confirm `hermes gateway status` shows connected.
2. Install the gateway as a service that survives reboots and restarts on crash: `hermes gateway install` (launchd on macOS, systemd on Linux).
3. Stop the machine sleeping. On macOS:

```
sudo pmset -a sleep 0
sudo pmset -a disksleep 0
pmset -g | grep sleep
```

The screen can still lock. Only system sleep has to be off.

4. Keep it on the charger. Apple Silicon Macs keep running with the lid closed while plugged in.
5. Test it: close the lid, wait a few minutes, message the bot from your phone.
6. Optional, for webhooks: `cloudflared tunnel --url http://localhost:PORT` gives a free public URL. It changes on every restart, so re-register webhooks each time, or use a VPS if you rely on them.

7. Choosing a VPS

Any Ubuntu 22.04 or 24.04 server with at least 1 vCPU works; 4 GB of RAM keeps multi-tool sessions smooth. Prices change often, so compare current plans rather than trusting a table.

- **Easiest start, best tutorials:** DigitalOcean, Linode, Vultr (Vultr has the most regions, useful for Southeast Asia).
- **Most RAM per dollar:** Contabo and Hostinger. Cheap tiers can share CPU, and intro prices often rise at renewal.

- **Free:** Oracle Cloud Always Free (ARM). Check every dependency has an ARM build; Hermes itself runs fine on ARM.
- **Already on AWS:** Lightsail for a fixed price. Raw EC2 only if you know AWS: forgotten volumes and data transfer can produce surprise bills.

8. Setting up the VPS

1. Create an Ubuntu 22.04 or 24.04 server. Add an SSH key during creation if the provider offers it. On EC2, allow inbound port 22 in the security group.

2. Connect: `ssh root@YOUR_SERVER_IP`

3. Make a non-root user so a misbehaving agent can do less damage:

```
adduser hermes && usermod -aG sudo hermes && su - hermes
```

4. Update and install the basics:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl git tmux libatomic1
```

5. Install Hermes and run `hermes setup` as in section 1.

6. Set up Telegram as in section 4, on the server this time, then `hermes gateway start`. It keeps running after you log out and across reboots.

7. Firewall: Telegram polling needs no inbound port. Only for webhooks:

```
sudo ufw allow 22/tcp && sudo ufw allow 443/tcp && sudo ufw enable
```

8. Test with your laptop closed: message the bot from your phone.

9. How MCP servers work

Hermes has a built-in MCP client. Each server's tools show up in the CLI, the Desktop app and Telegram, named `mcp_<server>_<tool>`.

- **From the catalog:** `hermes mcp catalog` to browse, `hermes mcp install <name>` to add one with guided sign-in.
- **By hand:** `hermes mcp add <name> --url "https://..." --auth oauth` for a hosted server, or `--command / --args / --env` for a local one.
- **Manage:** `hermes mcp list`, `hermes mcp test <name>`, `hermes mcp remove <name>`.

Local (stdio) servers do **not** inherit your shell environment. Only basics like `PATH` and `HOME` pass through, so list every key a server needs with `--env`. Restart Hermes after any MCP change; there is no hot reload.

Suki Pulse adds its own local server this way: `mcp-server/server.py` in the project repo exposes the `suki` tools to Hermes.

10. GitHub

1. Create a fine-grained personal access token at `github.com/settings/tokens` , limited to the repos Hermes should touch.
2. Put it in `~/.hermes/.env` as `GITHUB_TOKEN=...`
3. Register GitHub's hosted server:

```
hermes mcp add github --url "https://api.githubcopilot.com/mcp/" --auth header --env GITHUB_TOKEN
```

4. Restart, then `hermes mcp test github` , then ask the agent to list open issues on one of your repos.

Use `--auth oauth` instead for short-lived browser sign-in tokens. The old `@modelcontextprotocol/server-github` npm package is archived; do not use it. If `gh` is already signed in, it covers most repo work without any MCP server.

11. Vercel, Neon and Supabase

All three are in the catalog and follow the same four steps:

Service	Install	Sign in	Try
Vercel	<code>hermes mcp install vercel</code>	Browser OAuth	"List my Vercel projects"
Neon	<code>hermes mcp install neon</code>	OAuth or API key	"Run SELECT 1 on my main branch"
Supabase	<code>hermes mcp install supabase</code>	<code>hermes mcp login supabase</code> or a personal access token	"List my Supabase tables"

Restart Hermes and run `hermes mcp test <name>` after each. Neon and Supabase tools can change live data, so give them read-only access wherever you can.

12. Google Workspace

Not in the catalog, so it is added by hand. It covers Gmail, Calendar, Drive, Docs, Sheets and Slides.

1. In Google Cloud Console, create a project and enable the Gmail, Drive, Docs, Sheets, Calendar and Slides APIs.
2. OAuth consent screen: choose External and add your own Google account as a test user.
3. Credentials, then OAuth client ID, type Desktop app. Copy the client ID and secret.
4. Add both to `~/.hermes/.env` as `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` .

5. Register a Workspace MCP package, for example:

```
hermes mcp add google_workspace --command "npx" --args -y google-workspace-mcp --env
GOOGLE_CLIENT_ID GOOGLE_CLIENT_SECRET
```

Packages differ in tool names and env var names, so read the README of the one you pick. The config only stores the variable names; the values stay in `.env`.

6. Restart Hermes and sign in through the browser on first use. Ask for your recent emails or today's calendar to test.

Simpler alternative: the built-in `google-workspace` skill calls the Google APIs directly with a token Hermes manages, with no MCP server at all.

13. Final check

Check	Command
Overall health	<code>hermes doctor</code>
Every component	<code>hermes status --all</code>
MCP servers	<code>hermes mcp list</code> , then <code>hermes mcp test <name></code>
Gateway	<code>hermes gateway status</code> or <code>/platform list</code>
Live logs	<code>hermes logs -f</code>

You are done when `hermes` opens a working chat, `hermes desktop` answers, your Telegram bot replies while the laptop is closed (or from a VPS), and `hermes mcp list` shows every server you added as connected.